

School of Electrical and Computer Engineering

Technical University of Crete

Course: Optimization 2025-2026

Exercise 4

Support Vector Machines for Linearly Separable Data

Author: A. P. Liavas

December 9, 2025

In this exercise, we study the application of Support Vector Machines (SVMs) to a simple binary classification problem.

1 Binary classification with linearly separable data

We consider the **linearly separable** case. That is, we assume that there exist $\mathbf{0} \neq \mathbf{a}_* \in \mathbb{R}^n$ and $b_* \in \mathbb{R}$, such that the hyperplane

$$\mathbb{H}_{\mathbf{a}_*, b_*} := \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{a}_*^T \mathbf{x} = b_*\} \quad (1)$$

separates the data. Thus, for each data pair $(\mathbf{x}_i, y_i)$, for $i = 1, \dots, N$, we have

$$y_i = \begin{cases} +1, & \text{if } \mathbf{a}_*^T \mathbf{x}_i \geq b_*, \\ -1, & \text{if } \mathbf{a}_*^T \mathbf{x}_i < b_*. \end{cases} \quad (2)$$

2 Distance of a point from a hyperplane

We have proved that the distance of a point $\mathbf{x}$ from the hyperplane $\mathbb{H}_{\mathbf{a}, b}$ is given by

$$\mathcal{D}(\mathbf{x}, \mathbb{H}_{\mathbf{a}, b}) = \frac{|\mathbf{a}^T \mathbf{x} - b|}{\|\mathbf{a}\|_2}. \quad (3)$$

Thus, the margin, with respect to $\mathbb{H}_{\mathbf{a},b}$, is

$$\min_{i=1 \dots, n} \frac{|\mathbf{a}^T \mathbf{x}_i - b|}{\|\mathbf{a}\|_2}. \quad (4)$$

3 The SVM problem

We consider the problem

$$\begin{aligned} \max_{\mathbf{a}, b} \quad & \left\{ \min_{i=1 \dots, n} \frac{|\mathbf{a}^T \mathbf{x}_i - b|}{\|\mathbf{a}\|_2} \right\} \\ \text{s.t.} \quad & \mathbf{a}^T \mathbf{x}_i - b \geq 0, \text{ if } y_i = +1, \\ & \mathbf{a}^T \mathbf{x}_i - b < 0, \text{ if } y_i = -1, \end{aligned} \quad (5)$$

This problem is non-convex. We shall develop a convex reformulation, as follows.

First, we note that if $(\mathbf{a}, b)$ is an optimal solution of (5) and $\alpha \neq 0$, then $(\alpha \mathbf{a}, \alpha b)$ is also optimal for (5) (why?). We can therefore set

$$\min_{i=1 \dots, n} |\mathbf{a}^T \mathbf{x}_i - b| = 1. \quad (6)$$

Then, the problem becomes

$$\begin{aligned} \max_{\mathbf{a}, b} \quad & \frac{1}{\|\mathbf{a}\|_2} \\ \text{s.t.} \quad & \min_{i=1 \dots, n} |\mathbf{a}^T \mathbf{x}_i - b| = 1, \\ & \mathbf{a}^T \mathbf{x}_i - b \geq 0, \text{ if } y_i = +1, \\ & \mathbf{a}^T \mathbf{x}_i - b < 0, \text{ if } y_i = -1. \end{aligned} \quad (7)$$

This reformulation is equivalent to

$$\begin{aligned} \min_{\mathbf{a}, b} \quad & \|\mathbf{a}\|_2^2 \\ \text{s.t.} \quad & \min_{i=1 \dots, n} |\mathbf{a}^T \mathbf{x}_i - b| = 1, \\ & \mathbf{a}^T \mathbf{x}_i - b \geq +1, \text{ if } y_i = +1, \\ & \mathbf{a}^T \mathbf{x}_i - b < -1, \text{ if } y_i = -1, \end{aligned} \quad (8)$$

and, finally (why?), to

$$\begin{aligned} \min_{\mathbf{a}, b} \quad & \|\mathbf{a}\|_2^2 \\ \text{s.t.} \quad & \mathbf{a}^T \mathbf{x}_i - b \geq +1, \text{ if } y_i = +1, \\ & \mathbf{a}^T \mathbf{x}_i - b < -1, \text{ if } y_i = -1. \end{aligned} \quad (9)$$

4 A simplified variation of the SVM problem

We use `matlab` notation and define $\mathbf{w} := [b; \mathbf{a}]$. Furthermore, we augment the data as $\bar{\mathbf{x}}_i := [-1; \mathbf{x}_i]$. Then, the separation equation is given by

$$\mathbf{w}^T \bar{\mathbf{x}}_i \geq 0. \quad (10)$$

A slight variation of the SVM problem (9) is¹

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|_2^2, \quad \text{subject to } y_i \mathbf{w}^T \bar{\mathbf{x}}_i \geq 1, \quad i = 1, \dots, n. \quad (11)$$

Problem (11) is equivalent to

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|_2^2, \quad \text{subject to } 1 - y_i \mathbf{w}^T \bar{\mathbf{x}}_i \leq 0, \quad i = 1, \dots, n. \quad (12)$$

We define

$$f_0(\mathbf{w}) := \frac{1}{2} \|\mathbf{w}\|_2^2, \quad (13)$$

and, for $i = 1, \dots, n$,

$$f_i(\mathbf{w}) = 1 - y_i \mathbf{w}^T \bar{\mathbf{x}}_i. \quad (14)$$

Thus, the problem (11) can be expressed as

$$\min_{\mathbf{w}} f_0(\mathbf{w}), \quad \text{subject to } f_i(\mathbf{w}) \leq 0, \quad i = 1, \dots, n. \quad (15)$$

We have

$$\nabla f_0(\mathbf{w}) = \mathbf{w}, \quad \nabla^2 f_0(\mathbf{w}) = \mathbf{I}, \quad (16)$$

and

$$\nabla f_i(\mathbf{w}) = -y_i \bar{\mathbf{x}}_i, \quad \nabla^2 f_i(\mathbf{w}) = \mathbf{O}, \quad i = 1, \dots, n. \quad (17)$$

5 Barrier function for the simplified SVM problem

In order to apply the barrier method, we define the **barrier function**

$$\phi(\mathbf{w}) := - \sum_{i=1}^n \log(-f_i(\mathbf{w})). \quad (18)$$

¹Note that, in (11), we regularize b , which does not happen in (9). Usually, this has a small impact on the solution.

We have

$$\nabla \phi(\mathbf{w}) = \sum_{i=1}^n \frac{1}{-f_i(\mathbf{w})} \nabla f_i(\mathbf{w}) = \sum_{i=1}^n \frac{-y_i}{y_i \bar{\mathbf{x}}_i^T \mathbf{w} - 1} \bar{\mathbf{x}}_i = \sum_{i=1}^n \frac{y_i}{1 - y_i \bar{\mathbf{x}}_i^T \mathbf{w}} \bar{\mathbf{x}}_i, \quad (19)$$

and

$$\nabla^2 \phi(\mathbf{w}) = \sum_{i=1}^n \frac{1}{f_i^2(\mathbf{w})} \nabla f_i(\mathbf{w}) \nabla f_i(\mathbf{w})^T = \sum_{i=1}^n \frac{1}{(y_i \bar{\mathbf{x}}_i^T \mathbf{w} - 1)^2} \bar{\mathbf{x}}_i \bar{\mathbf{x}}_i^T. \quad (20)$$

6 Barrier method for the simplified SVM problem

We shall use the Newton method and minimize the cost functions

$$g_k(\mathbf{w}) := t_k f_0(\mathbf{w}) + \phi(\mathbf{w}), \quad (21)$$

for an increasing sequence of coefficients t_k , for $k = 1, \dots, K$.

We have

$$\nabla g_k(\mathbf{w}) = t_k \nabla f_0(\mathbf{w}) + \nabla \phi(\mathbf{w}) = t_k \mathbf{w} + \sum_{i=1}^n \frac{y_i}{1 - y_i \bar{\mathbf{x}}_i^T \mathbf{w}} \bar{\mathbf{x}}_i, \quad (22)$$

and

$$\nabla^2 g_k(\mathbf{w}) = t_k \nabla^2 f_0(\mathbf{w}) + \nabla^2 \phi(\mathbf{w}) = t_k \mathbf{I} + \sum_{i=1}^n \frac{1}{(y_i \bar{\mathbf{x}}_i^T \mathbf{w} - 1)^2} \bar{\mathbf{x}}_i \bar{\mathbf{x}}_i^T. \quad (23)$$

6.1 Newton step

The Newton step for the minimisation of g_k is given by

$$\mathbf{w}^+ = \mathbf{w} - s (\nabla^2 g_k(\mathbf{w}))^{-1} \nabla g_k(\mathbf{w}), \quad (24)$$

where s is computed using backtracking for the following reasons:

1. first, we must guarantee that the new point is feasible, that is, $\mathbf{w}^+ \in \mathbf{dom} \phi$;
2. second, we must perform backtracking line search for the function g_k .

In this exercise, the following `matlab` files are provided:

1. `separable_SVMs_interior_point_method.m` (main file)
2. `[X, y] = generate_data(N, a, b)` (data generating function)
3. `flag = data_is_not_separable(X, y, a, b)` (returns 0 if data is separable, with respect to `a` and `b`, and 1 otherwise).

You are asked to implement the following functions:

1. `flag = point_is_feasible(w_init, X_augm, y)`, which returns 1 if $w_{\text{init}} \in \text{dom}\phi$ and 0 otherwise.

Answer:

The code of the function is as follows:

```
function flag = point_is_feasible(w_init,X_augm,y)
flag = 1;

N = length(y);

tmp = y' .* (X_augm' * w_init) > 1;
if ( sum(tmp) < N )
    flag = 0;
end

end
```

2. `grad = gradient_SVM_barrier(w(:,inner_iter), X_augm, y, t)`, which computes the gradient of the cost function g_k at the point $w(:, \text{inner_iter})$.

Answer:

The code of the function is as follows:

```
function grad = gradient_SVM_barrier(w_inner_iter,X_augm,y,t)

N = length(y);
sum = 0;
for i = 1:N
    scal = y(i) / (1 - y(i) .* X_augm(:,i)' * w_inner_iter);
    sum = sum + scal .* X_augm(:,i);
end
grad = t*w_inner_iter + sum;
end
```

3. `hess = Hess_SVM_barrier(w(:,inner_iter), X_augm, y, t)`, which computes the Hessian of the cost function g_k at the point $w(:,inner_iter)$.

Answer:

The code of the function is as follows:

```
function hess = Hess_SVM_barrier( w_inner_iter, X_augm, y, t )

N = length(y);
sum = 0;

for i = 1:N
    scal = 1 / (1 - y(i) .* X_augm(:,i)' * w_inner_iter)^2;
    sum = sum + scal .* X_augm(:,i) * X_augm(:,i)';
end

hess = t*ones(length(X_augm(:,1))) + sum;

end
```

4. `barrier_SVM_cost_function(w_new, X_augm, y, t)`, which computes the value of g_k at the point w_new .

Answer:

The code of the function is as follows:

```
function g_val = barrier_SVM_cost_function(w_new,X_augm,y,t)

f_0 = (1/2).*norm(w_new)^2;
phi = -sum(log(y' .* (X_augm' * w_new) - 1));
g_val = t*f_0 + phi;

end
```

Upon executing the main script we get the figures that are attached. The first plot includes the initial separating hyperplane that is drawn by randomly generating the vectors $\mathbf{a}$ and $\mathbf{b}$ and then using them to create the initial separating hyperplane (2D line).

After that we obtain the optimal pair of $\mathbf{a}$ and b (concatenated in one vector $\mathbf{w} = [\mathbf{a}; b]$) by solving the corresponding SVM problem, via CVX.

An instance of the two separating hyperplanes as well as the randomly generated 2D data is given below:

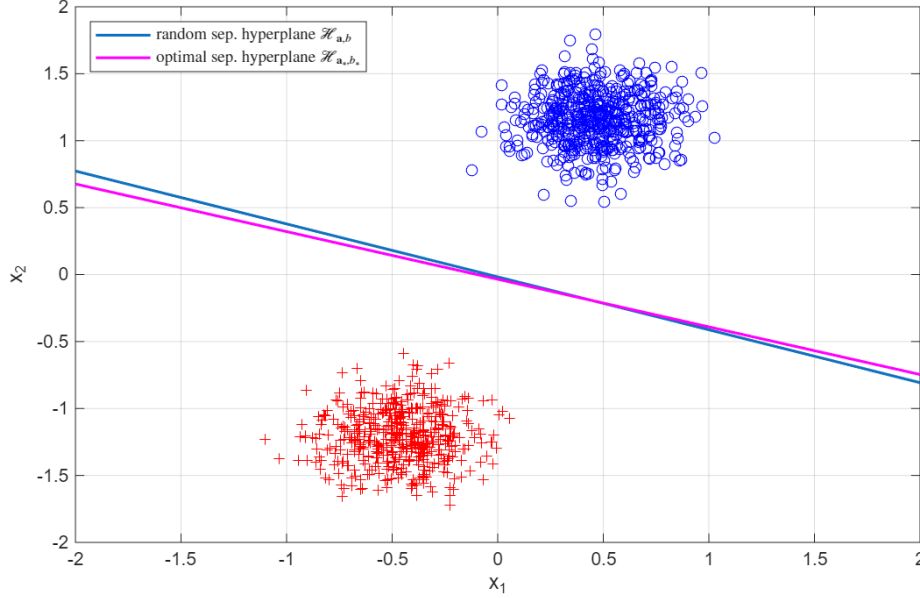
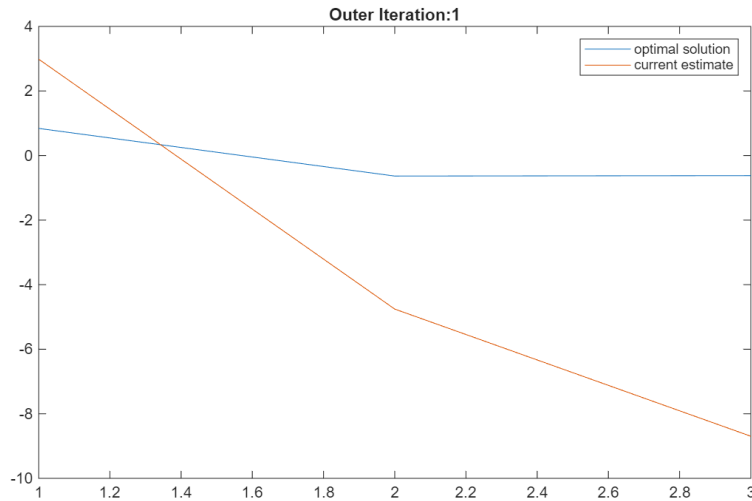
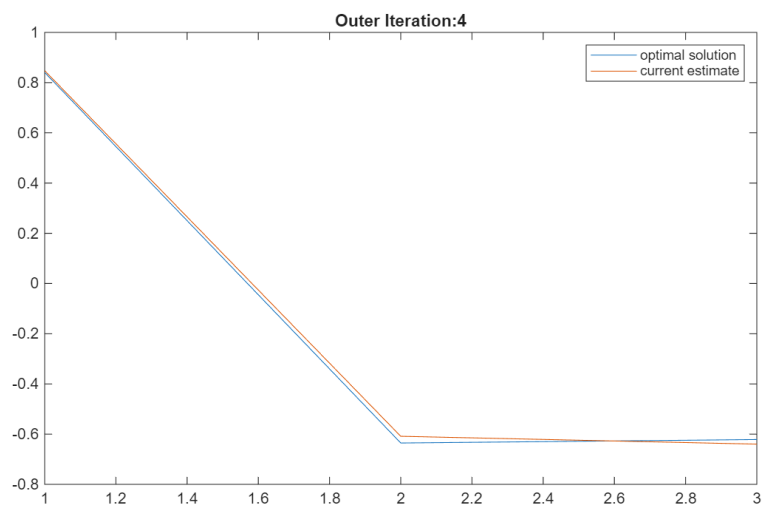
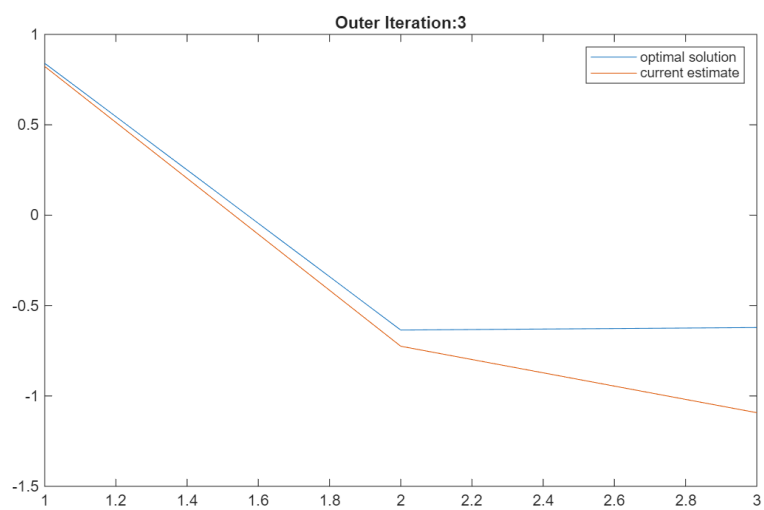
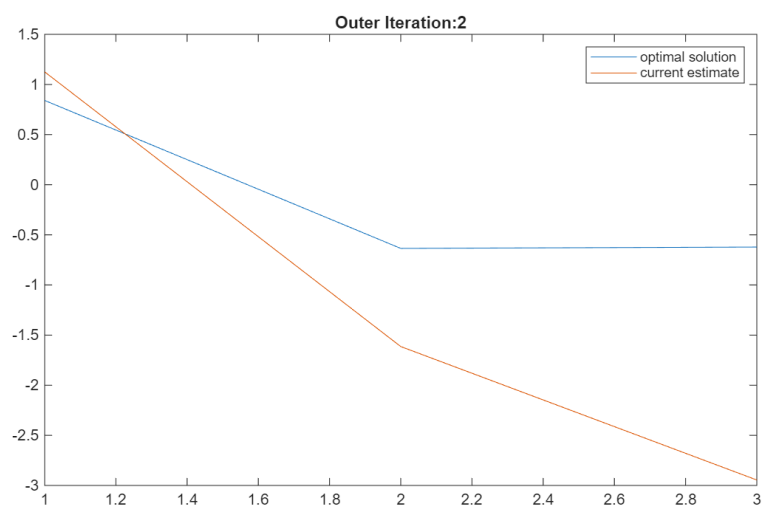
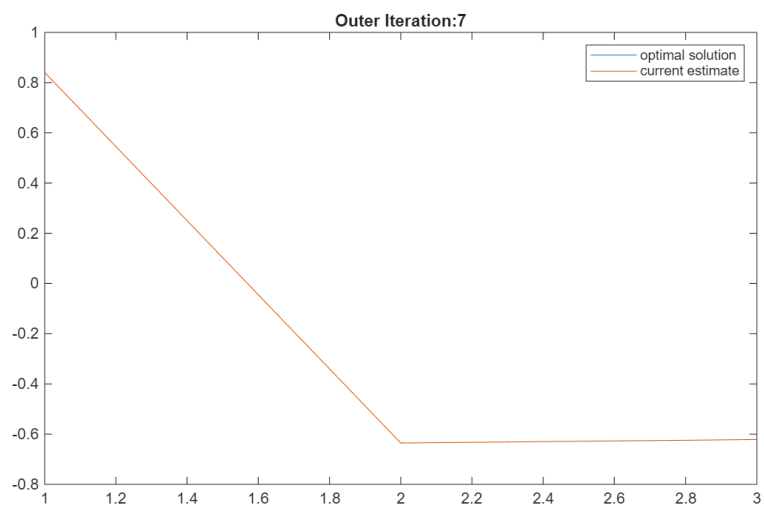
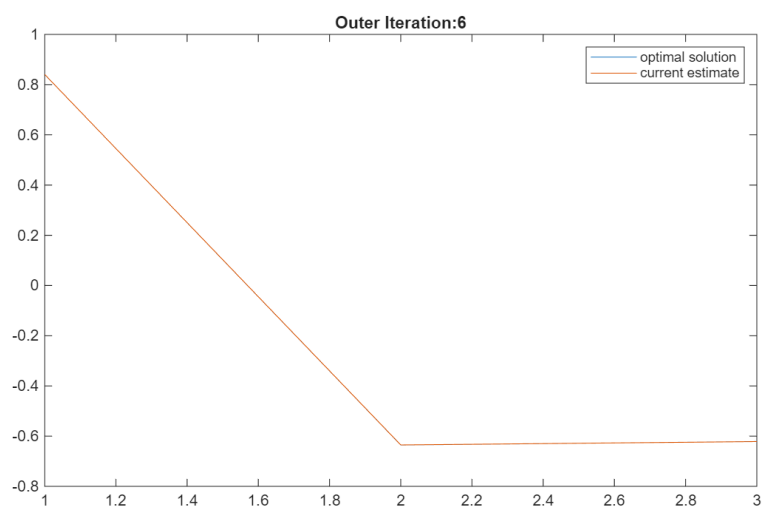
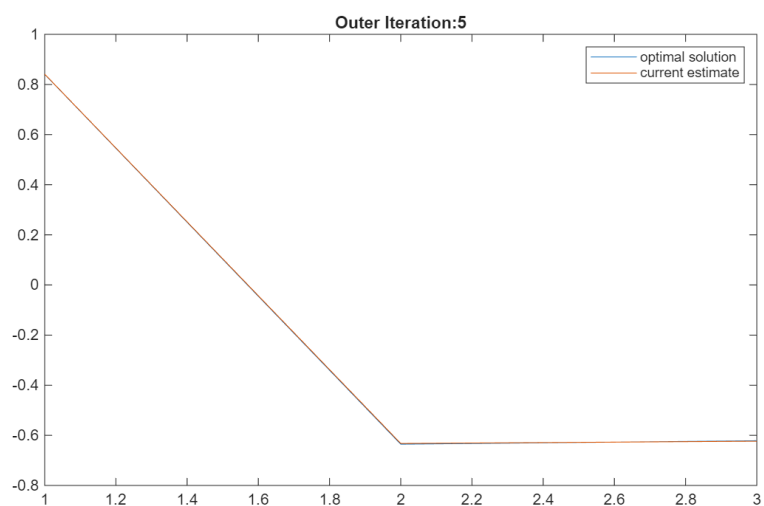


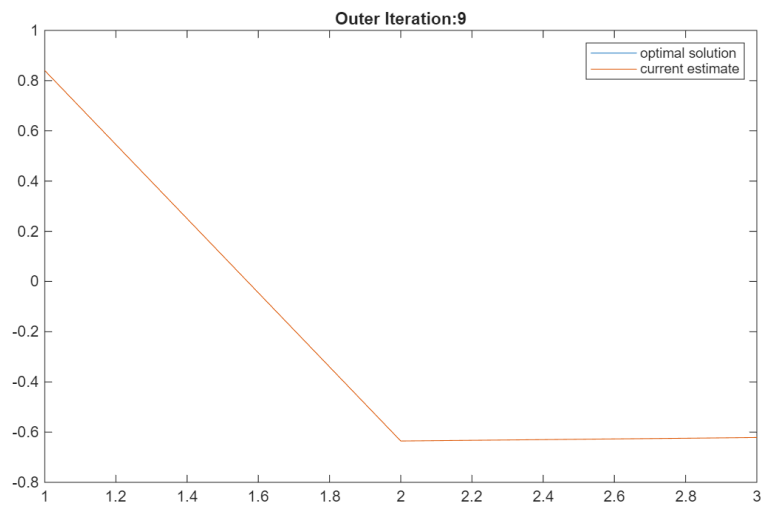
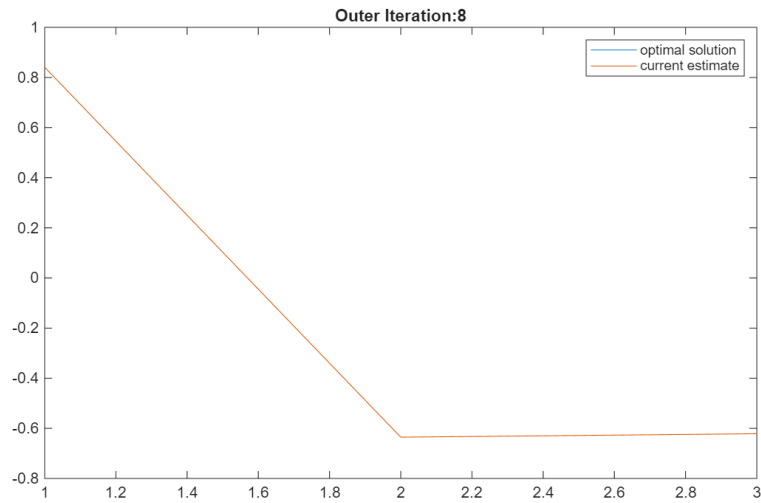
Figure 1: Datapoints in the 2D plane and the separating lines (initial-blue and optimal-magenta)

Lastly, we use the *Interior Point Method* to solve the SVM problem and obtain the optimal value for the vector $\mathbf{w} = [\mathbf{a}; b]$. In each iteration, we plot the current estimate of the solution that the algorithm gives in comparison to the optimal 3-tuple $[a_{*1} \ a_{*2} \ b_*]$, computed by CVX, earlier. Below, are all the estimation stages that the *Interior Point Method* went through before finding the solution (that is, getting ϵ -close to it).









As we can easily verify, the algorithm tends to overlap with the optimal solution pretty fast, finding the solution in just under 10 iterations, for this instance of the problem.

In fact, the difference between the orange and the blue line is almost negligible already from the 5th iteration, since there is no way to visually distinguish them, unless we zoom in, using the `matlab` tools.